

Assignment 4 Reflection - Ben Branta

1. Describe the practice of looping over containers in Python, particularly compared to other experiences you've had, such as Excel or R.

Answer: Looping over a list, tuple, or dictionaries in Python is like opening the container and pulling out one item at a time to do something with it. Compared to Excel my most common solution is to import into Python. In my opinion Python is far more flexible and useful. However, I don't use Excel for these types of problems and I can imagine it might be useful for a simple use case where the overhead of setting up Python is too much. When using R I found R was flexible and useful like Python. The syntax with R was less readable in my opinion. With R I found its built-in vectorized functions or methods were more efficient and easier to read than iterative loops. I think we will use vectorization with pandas and Python. I don't have much experience using apply functions with pandas and Python so I look forward to that comparison with R later.

2. Do some research. What is None in Python? Why is it the default returned value from functions?

Answer: None is a way for Python to say this object has no value. Its an instance of NoneType which makes it a convenient way to check if a function returns something. It is the default returned value because functions have to evaluate to an object. If the function does not explicitly return something the function will return None.

3. Describe the experimentation you did for the sequences of fizzbuzz for collatz sequences. What did you do? What are your observations?

Answer: I tried to answer the question, "What's the first integer n for which collatz_fizzbuzz(n) contains m 'fizz's?" I would return the count saved in the 'fizz' key from the previous function, collatz_fizzbuzz_counts, while incrementing n, until m equalled the number of 'fizz'. I didn't know where else to go with this equation to be honest. I could increase m and I found that the minimum value for n appeared to double.

4. What was the most difficult part of the text analysis problem, and why? How did you overcome it?

Answer: I am familiar with nesting functions so that made the text analysis problem smooth. However, in these cases I can become overwhelmed by many nested functions. One way I overcome it is by writing single purpose functions with clear names and that helped a lot. It helps to write docstrings for these functions. I didn't do it this time due to a time constraint. Then using IDE's like VS Code allows me to hover over defined functions and read them without having to jump back and forth in the code.

5. Name a programming mistake you found yourself making a lot at first in this class, and which you make a lot less now. How did you reduce the number of instances of this mistake, and how might you approach novel mistakes to help your recognize them faster in the future?

Answer: I usually come to a solution that works and stop thinking about other ways to do it. In this course I've found that my familiarity with these topics allows me to focus less on the syntax and more on the implementation. These problems allow me to think more about other ways to do them which helps build my toolkit for future problem solving.

6. Describe one thing that you learned while working on this lesson that stood out as useful or interesting.

Answer: I like how the text analysis problem created helper functions and then continued to build on them. The idea of breaking a problem down to its frame and creating the building blocks to solve it stands out as the most interesting thing in this lesson for me.